

Plan de Gestión de la Configuración de Software

Proyecto FinanceSmart

Versión 1.0 · 3 de mayo de 2026
Repositorio GitHub — FinanceSmart

Plan de Gestión de la Configuración de Software (SCMP)

Proyecto: FinanceSmart

Versión: 1.0 Fecha: 2026-05-03 Repositorio: GitHub — FinanceSmart Rama principal: main

1. Aplicación Metodología y Gestión de Proyectos (M1)

1.1 Introducción

Propósito del SCMP

Este documento define cómo el equipo de FinanceSmart identifica, controla, audita y libera todos los elementos de configuración del software (código fuente, documentación, dependencias, datos de prueba, infraestructura como código y artefactos de despliegue). Su finalidad es:

- Garantizar que cualquier versión del producto sea **reproducible** por cualquier integrante del equipo.
- Evitar regresiones mediante un flujo de control de cambios disciplinado.
- Dar trazabilidad entre requerimientos del socio formador, commits, pruebas y releases.
- Proteger la calidad del producto entregado al socio formador en cada sprint.

Alcance

Incluye:

- Código fuente del frontend (React 19 + Vite) en frontend/ .
- Código fuente del backend (Node.js + Express 5) en backend/ .
- Esquemas y migraciones de base de datos Supabase (PostgreSQL).
- Scripts de seed (backend/scripts/seed-sofia.js).
- Pruebas unitarias (Jest), de integración y de regresión contra LLM real.
- Configuración de despliegue en Vercel (vercel.json).
- Workflows de CI/CD en .github/workflows/ .
- Documentación técnica (docs/ , SEED_SOFIA_README.md).
- Variables de entorno y secretos (gestionados fuera del repo).

No incluye:

- Datos personales reales de usuarios productivos (sólo cuentas demo como Sofía).
- Infraestructura externa de Supabase (gestionada por su panel).
- Llaves de API de proveedores LLM (Gemini, Ollama) — sólo se documentan referencias.

Definiciones clave

Término	Definición
Elemento de configuración (CI)	Cualquier artefacto bajo control de versiones que afecte el comportamiento del sistema.
Baseline	Versión etiquetada y aprobada del producto, lista para despliegue o demo al socio.
Pull Request (PR)	Mecanismo de propuesta de cambio en GitHub que activa revisión + CI.
Sprint	Iteración de 2 semanas alineada con el calendario académico del Reto.
Release	Despliegue a producción tagueado con versión semántica (ej. <code>v1.2.0</code>).
Hotfix	Cambio urgente directo sobre <code>main</code> que corrige un defecto crítico en producción.
Regresión	Suite de prompts + assertions que valida que el chatbot no alucine ni rompa contratos.

1.2 Gestión de la Configuración

Roles y responsabilidades del equipo

Integrante	Rol primario	Responsabilidades de configuración
Oscar	Líder técnico / Backend & Agentic AI	Aprueba PRs al backend; mantiene el contrato de tools del agente; gestiona la rama <code>main</code> ; etiqueta releases.
Jorge	Frontend / Despliegue Web	Mantiene <code>frontend/</code> ; aprueba PRs de UI; configura Vercel y monitorea builds de producción.
Elvia	Reto / Integración IA	Custodia de prompts, fixtures y suite de regresión; coordina demos con el socio.
Frumen	Diseño	Custodia de wireframes y guía visual; valida cambios de UX antes de merge.
Iliana	Calidad	Define criterios de aceptación; revisa cobertura de pruebas (Jest + Vitest); audita conformidad.
Luis	Base de datos	Custodia del esquema PostgreSQL; revisa toda migración; valida normalización (3NF).

Cualquier merge a `main` requiere aprobación del responsable del área tocada (backend → Oscar, frontend → Jorge, BD → Luis, etc.).

Políticas de control de cambios

- 1. **Toda modificación pasa por Pull Request.** Está prohibido empujar directamente a `main` .
- 2. **Convención de commits:** encabezado descriptivo en imperativo (`Fix: ...` , `Add: ...` , `UI: ...` , `Refactor: ...`). Mensajes en español o inglés, consistentes dentro del PR.
- 3. **Mínimo 1 revisor** por PR antes de merge; **2 revisores** si toca esquema BD o flujo de pago/auth.

- 4. **CI obligatoria en verde** (`ci.yml`) antes de habilitar el botón de merge.
- 5. **Cambios destructivos** (drop de tablas, borrado de datos) requieren respaldo previo y aprobación del líder técnico.
- 6. **Secretos jamás se commitean**. Se usan `.env` locales y Vercel Environment Variables en producción. El repo incluye `.env.example` como plantilla.

Herramientas seleccionadas

Categoría	Herramienta	Justificación
Control de versiones	Git + GitHub	Estándar de la industria; integración nativa con Vercel y Actions.
Gestión de tareas	GitHub Projects / Issues	Trazabilidad ticket↔commit↔PR sin herramienta extra.
CI/CD	GitHub Actions + Vercel	Automatización gratuita; despliegue cero-config en Vercel.
Pruebas	Jest (backend), Vitest (frontend planeado), suite de regresión propia	Estándar Node; regresión propia para alucinaciones del LLM.
Comunicación	WhatsApp (operativa) + Sesiones de Reto (formales)	Bajo overhead; sesiones documentadas para acta.
Base de datos	Supabase (Postgres)	PaaS con auth, RLS y dashboard incluidos.

1.3 Actividades de Configuración

Identificación de elementos controlados

Tipo	Ubicación	Política
Código backend	backend/**/*.*js	Versionado completo; PR obligatorio.
Código frontend	frontend/src/**	Versionado completo; PR obligatorio.
Migraciones BD	backend/migrations/*.sql	Inmutables una vez en main ; cambios sólo vía nueva migración.
Seed de demo	backend/scripts/seed-sofia.js	Versionado; cambios documentados en SEED_SOFIA_README.md .
Pruebas	backend/__tests__/, backend/regression/	Toda PR que cambie código de negocio debe ajustar/añadir pruebas.
Documentación	docs/ , *.md raíz	Versionada; revisada en PR.
Configuración despliegue	vercel.json , .github/workflows/*.yaml	Cambios revisados por líder técnico + responsable de despliegue.
Dependencias	package.json + package-lock.json	npm install siempre con --save-exact o respetando rango ya definido; lock se commitea.
Variables de entorno	.env.example (plantilla), Vercel UI (valores)	Valores reales nunca en repo.

Flujo de control de cambios

1. Crear issue (descripción + criterios de aceptación)
↓
2. Crear rama desde main: feature/<short-name> | fix/<bug-id> | chore/<task>
↓
3. Desarrollar + commits atómicos + pruebas locales (npm test)
↓
4. Push a origin → abrir Pull Request
↓
5. CI ejecuta: lint + jest + build (.github/workflows/ci.yml)
↓ (si falla)

✅ verde

❌ corregir + re-push

↓
6. Revisión por par (mínimo 1 aprobación)
↓
7. Merge a main (squash-merge preferido para historial limpio)
↓
8. Despliegue automático a Vercel (preview en PR; producción en main)
↓
9. Cierre del issue con referencia al commit/PR

Seguimiento de estado (versionado)

- **Esquema SemVer:** MAJOR.MINOR.PATCH (v1.2.3).
 - MAJOR : cambio incompatible en API pública (ej. cambio de contrato del endpoint /api/chatbot).
 - MINOR : nueva funcionalidad retrocompatible (ej. nuevo widget del chatbot).
 - PATCH : corrección de bug sin cambio de comportamiento.
- **Tags Git** marcan cada release: `git tag -a v1.2.0 -m "Sprint 4 release"`.
- **CHANGELOG.md** se actualiza por release con sección Added / Changed / Fixed / Removed .
- **Branch protection:** main requiere PR + status checks + 1 review.

Auditorías

Frecuencia	Tipo de auditoría	Responsable	Verifica
Cada PR	Revisión de código	Revisor asignado	Cumplimiento de convenciones, pruebas, seguridad básica.
Semanal	Revisión de issues abiertos vs. PRs	Líder técnico	Que ningún cambio quede huérfano sin merge ni cierre.
Por sprint	Auditoría de baseline	Calidad (Iliana)	Que la versión etiquetada para demo corresponda exactamente al commit en <code>main</code> y que CI haya pasado.
Por release	Auditoría de configuración	Líder técnico + Calidad	Que <code>package-lock.json</code> , migraciones aplicadas y variables de entorno en Vercel estén alineadas con el release.
Mensual	Auditoría de dependencias	Backend / Frontend	<code>npm audit</code> + revisión de licencias y CVEs.

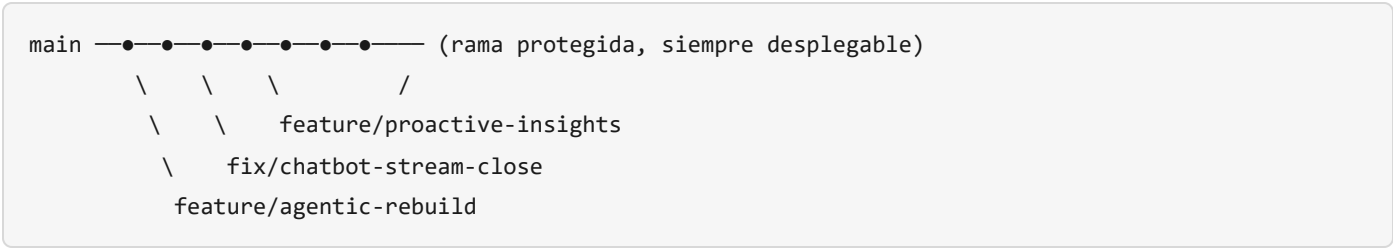
1.4 Herramientas y Recursos

Herramientas específicas

- **Git** $\geq$ 2.40 (cliente local).
- **GitHub** (repositorio remoto, Actions, Issues, Projects).
- **GitHub Actions** (CI/CD) con runners ubuntu-latest.
- **Vercel** (despliegue frontend + funciones serverless del backend).
- **Supabase Dashboard** (administración de BD, auth, storage).
- **Node.js 20 LTS** (entorno de ejecución).
- **npm** (gestor de paquetes; `package-lock.json` versionado).
- **Jest 30** (pruebas backend).
- **ESLint + Prettier** (calidad de código frontend).

Estrategia de ramas

Se sigue una variante simplificada de **trunk-based + feature branches**:



Tipo de rama	Patrón	Vida útil	Origen	Destino
Feature	feature/<nombre>;	Hasta merge	main	main (PR)
Bugfix	fix/<id-o-nombre>;	Hasta merge	main	main (PR)
Chore	chore/<tarea>;	Hasta merge	main	main (PR)
Hotfix	hotfix/<id>;	Horas	main (tag de prod)	main (PR urgente)
Experimentos	spike/<idea>;	Sin garantía de merge	main	descartable

Ramas históricas activas en este proyecto: `Oscar` , `AgentTest2` (rebuild agentic). Tras cierre de cada sprint las ramas mergeadas se eliminan.

Normas de uso

- **Nombre de rama** en kebab-case, prefijado por tipo.
- **Rebase preferido sobre merge** para mantener historial lineal cuando la rama es propia.
- **No commitear binarios pesados** (>10 MB). Usar Vercel Blob o Supabase Storage.
- **No commitear** `.env` , `node_modules/` , `dist/` , claves o tokens.
- **Co-autoría documentada** con `Co-Authored-By:` cuando aplica.

Infraestructura

Capa	Plataforma	Notas
Frontend	Vercel (Static + Edge)	Despliegue automático por commit a <code>main</code> .
Backend (API)	Vercel Functions (Node.js, Fluid Compute)	Definido en <code>api/index.js</code> enrutando a Express.
Base de datos	Supabase Postgres	RLS habilitado; backups automáticos diarios del proveedor.
Almacenamiento de imágenes	Vercel Blob (planeado)	Para futuros adjuntos de movimientos.
LLM principal	Google Gemini 2.5 Flash vía API	Llave en variable <code>GEMINI_API_KEY</code> .
LLM fallback	Ollama (<code>qwen2.5:7b</code>) self-hosted	Sólo desarrollo/demo offline.
CI	GitHub Actions (<code>.github/workflows/ci.yml</code>)	Lint + Jest + build en cada PR.
Regresión LLM	GitHub Actions programado (<code>regression.yml</code>)	Semanal; consume cuota Gemini real.

1.5 Plan de Liberación

Estrategia de versionado

- **SemVer** (`MAJOR.MINOR.PATCH`).
- Pre-releases del socio formador se etiquetan `vx.Y.Z-sprintN` (ej. `v0.5.0-sprint4`).
- Cambios breaking se anuncian con antelación de 1 sprint en CHANGELOG.

Frecuencia de releases

- **Por sprint** (cada 2 semanas) coincidiendo con la demo al socio formador.
- **Hotfixes** según necesidad (incidentes críticos en producción).

Proceso de liberación paso a paso

1. **Congelar funcionalidad** 2 días antes del fin de sprint (sólo bugfixes después).
2. **Verificar CI verde** en `main` .
3. **Ejecutar suite completa**: `npm test` (backend), build de frontend, smoke test manual de flujos críticos (login, chatbot, dashboard).
4. **Ejecutar regresión LLM**: `cd backend && npm run regression` — exigir ≥ 95 % de pass rate.
5. **Actualizar** `CHANGELOG.md` con cambios desde el último tag.
6. **Crear tag**: `git tag -a vx.Y.Z -m "Sprint N"` y `git push --tags` .
7. **Vercel despliega automáticamente** desde `main` . Verificar URL de producción.
8. **Smoke test post-deploy** en URL de producción (login → chatbot → dashboard).
9. **Anuncio al socio formador** con liga, notas de versión y demo agendada.

Criterios de calidad (puerta de release)

Criterio	Umbral
Pruebas Jest	100 % verdes (181/181 actualmente)
Cobertura mínima módulos críticos	70 % en <code>routes/chatbot.js</code> , <code>utils/agent/</code> , <code>utils/tools/</code>
Regresión LLM (Gemini real)	≥ 95 % pass rate
Tasa de alucinación en grounded queries	0 %
Build de frontend	Sin errores ni warnings críticos
<code>npm audit</code> (high/critical)	0 vulnerabilidades sin parche disponible
Smoke test manual	Login, dashboard, chatbot (consulta + acción), demo Sofía

1.6 Control de Interfaces

Interfaces internas (frontend ↔ backend)

Endpoint	Verbo	Propósito	Versionado
/api/auth/login	POST	Autenticación	v1
/api/movimientos	GET/POST	CRUD movimientos financieros	v1
/api/metast	GET/POST/PATCH	Metas de ahorro	v1
/api/presupuestos	GET/POST	Presupuestos por categoría	v1
/api/recurrencias	GET/POST/PATCH	Suscripciones / cargos recurrentes	v1
/api/insights	GET	Insights deterministas	v1
/api/proactive-insights	GET	Insights generados por LLM (cache 6h)	v1
/api/chatbot	POST (SSE)	Stream agentic del chatbot	v1
/api/chatbot/confirm-action	POST	Confirmación de acción de escritura propuesta	v1
/api/chatbot/cancel-action	POST	Cancelación de propuesta	v1

Documentación: Cada endpoint documentado en su archivo `routes/*.js` con comentario de cabecera (request/response shape). El contrato del SSE del chatbot se documenta en `docs/CHATBOT_SSE.md` (planeado).

Interfaces externas (APIs de terceros)

Servicio	Uso	Documentación
Supabase	BD + Auth	https://supabase.com/docs
Google Gemini API	LLM principal	https://ai.google.dev/docs
Ollama	LLM local fallback	https://ollama.com/library
Vercel	Hosting + Functions	https://vercel.com/docs

Proceso ante cambios de interfaz

- Cambio breaking:** nuevo endpoint con sufijo `/v2/` y deprecación documentada del v1 con plazo de 1 sprint.
- Cambio aditivo retrocompatible:** documentado en CHANGELOG, sin nueva versión de endpoint.
- Cambio en proveedor externo:** se evalúa impacto, se ajusta adaptador (`utils/providers/*` , `aiProvider.js`) y se cubre con prueba.

Riesgos asociados

Riesgo	Mitigación
Gemini cambia formato de respuesta de tool calls	<code>zodToGeminiSchema</code> + validación zod estricta; pruebas de regresión semanales.
Supabase cambia política de RLS	Pruebas de integración con seed Sofía; backups antes de cada migración.
Vercel cambia runtime por defecto	Pin explícito en <code>vercel.json</code> .
Ollama no disponible en demo offline	Bandera de capacidad y mensaje "modo limitado" en UI.

1.7 Gestión de Proveedores

Dependencias externas principales

Backend (`backend/package.json`):

- `@supabase/supabase-js` ^2.103
- `express` ^5.2
- `@google/generative-ai` (Gemini SDK)
- `zod` ^4.x (validación de tools y respuestas estructuradas)
- `cors` , `dotenv` , `express-rate-limit`
- `jest` (dev)

Frontend (`frontend/package.json`):

- `react` ^19, `react-dom` ^19
- `vite` (build)
- `recharts` (gráficos del chatbot)
- `@supabase/supabase-js`

Servicios:

- Supabase (Postgres, Auth)
- Google Gemini (LLM)
- Vercel (hosting)
- GitHub (repo + CI)

Control de versiones externas

- `package-lock.json` se versiona obligatoriamente — garantiza instalación reproducible.
- Versiones se fijan con rango caret (`^`) por defecto; se fija exacta cuando hay incompatibilidades conocidas (ej. `zod@4.x` por uso de `z.toJSONSchema()`).
- Dependencias mayores se actualizan sólo dentro de un PR de tipo `chore/upgrade-<dep>` con pruebas verdes.

Políticas de actualización

Tipo de cambio	Política
Patch (X.Y.Z → X.Y.Z+1)	Aceptable en cualquier PR si CI sigue verde.
Minor (X.Y.0 → X.Y+1.0)	PR dedicado; revisar changelog del proveedor.
Major (X.0.0 → X+1.0.0)	PR dedicado + spike de validación; aprobación del líder técnico.
Vulnerabilidad crítica	Hotfix prioritario en ≤ 48 h.

Riesgos

Riesgo	Probabilidad	Mitigación
Cuota agotada en Gemini	Media	Suite de regresión gateada, fallback a Ollama, monitoreo de uso.
Cambio breaking en Express 5 (recién estable)	Media	Pruebas de integración cubren rutas críticas; pin de versión.
Licencia incompatible en nueva dependencia	Baja	Revisión manual de licencia (preferimos MIT/Apache 2.0/BSD).
Caída de Vercel	Baja	Documentación de despliegue alternativo; status page monitoreada.
Caída de Supabase	Baja	Backups diarios; plan documentado de recuperación.
Deprecación de modelo Gemini 2.5 Flash	Media	Capa de proveedor abstrae el modelo; cambio aislado a un archivo.

2. Documentación de Diseño (M2)

Responsable: Frumen

Avance de diseño correspondiente a los sprints completados. Incluye:

- Wireframes de pantallas principales (Login, Dashboard, Chatbot, Metas, Presupuestos).
- Guía visual: paleta de colores (rojos / verdes / naranjas / grises / blancos), tipografía, espaciado.
- Diagrama de componentes del frontend.
- Flujos de usuario (golden path: login → consulta chatbot → confirmar acción).

Nota: Este avance no se califica en la entrega parcial; se recibe retroalimentación del profesor.

3. Producto Funcional y Código (M3)

Responsables: Jorge (Web), Elvia (Reto), Luis (BD), Elvia (IA)

3.1 Repositorio

- URL: GitHub — `FinanceSmart`
- Rama productiva: `main`
- Última versión etiquetada: pendiente de tag inicial post-rebuild.

3.2 Sistema funcional al momento

- ☒ **Frontend** (React 19 + Vite) en producción en Vercel.
- ☒ **Backend** (Express 5 sobre Vercel Functions) en producción.
- ☒ **Conectividad frontend** ↔ **backend** vía REST + SSE para el chatbot.
- ☒ **Conectividad backend** ↔ **Supabase** con auth y RLS.

3.3 Implementación de Base de Datos

Responsable: Luis

Tablas principales:

- `usuarios` (gestionada por Supabase Auth + tabla extendida `usuario_perfil`)
- `categorias`
- `movimientos`
- `metas_ahorro`
- `presupuestos`
- `recurrencias`
- `tarjetas`
- `proactive_insights_cache` (cache de insights LLM, TTL 6h)

Normalización (3FN):

- 1FN: todos los campos atómicos (sin listas, sin JSON salvo en `parametros_json` justificado).
- 2FN: PKs simples (`id_*`); ningún atributo depende de subconjunto de PK compuesta (no hay PKs compuestas en tablas transaccionales).
- 3FN: ningún atributo no clave depende transitivamente de la PK. Ejemplo: en `movimientos` , `id_categoria` referencia `categorias(id_categoria)` ; el nombre de categoría NO se duplica en `movimientos` .

Excepciones documentadas:

- `proactive_insights_cache.observaciones_json` (JSONB) — desnormalizado intencionalmente porque es payload de salida del LLM consumido tal cual por el frontend; nunca se consulta por campos internos.

3.4 Integración IA

Responsable: Elvia

- Capa de proveedor (`backend/utils/aiProvider.js`) con dos implementaciones: **Gemini 2.5 Flash** (producción) y **Ollama qwen2.5:7b** (offline/dev).
- Selección por capacidad (`capabilities.supportsTools`): si el proveedor soporta tools → loop agentic; si no → context-injection.

- **11 herramientas** registradas (`backend/utils/tools/`):

- Lectura: resumen mes, movimientos, gastos por categoría, metas, presupuestos, recurrencias, tarjetas, salud financiera, insights. - Escritura (sólo emiten propuestas): aporte a meta, crear/modificar presupuesto, toggle recurrencia, crear meta.

- Validación zod estricta por herramienta con enums por usuario (categorías, tarjetas, metas) — **previene alucinación de IDs**.
- 11 widgets visuales emitidos por el chatbot: chart, simulator, streak, compare, gauge, heatmap, subs, top merchants, savings rate, recurring calendar, sparklines.
- **Suite de regresión** (`backend/regression/`) con 64 prompts contra Gemini real → 96.9 % pass rate, 0 % alucinación en consultas grounded.

4. Calidad (M4)

Responsable: Iliana

- **181 pruebas Jest** en backend, todas verdes.
- Cobertura de módulos críticos:
 - `routes/chatbot.js` - `utils/agent/loop.js` - `utils/agent/proposalStore.js` - `utils/tools/` - `utils/providers/`
- Suite de regresión con 11 tipos de assertion: `contains_number` , `cites_category` , `no_fabricated_categories` , `no_dates_outside_data` , `no_fabricated_amounts` , `invokes_tool` , `does_not_invoke_tool` , `refuses_gracefully` , `mentions_meta` , `language` , `max_length` .
- Reportes generados: `regression/report-*.json` , `regression/summary.csv` (commiteado para diff en PR).

Detalle adicional en el Módulo de Calidad (entrega individual y de equipo).

5. Deployment (M5)

Responsables: Jorge (Web), Elvia (Reto)

CI/CD activo

- `.github/workflows/ci.yml` — corre en cada PR y push a `main` :
 - Instalación con `npm ci` .
 - Lint del frontend.
 - Pruebas Jest del backend.
 - Build del frontend.
- `.github/workflows/regression.yml` — programado semanal + manual:
 - Ejecuta suite de regresión LLM contra Gemini real.
 - Publica reporte como artifact.
- **Vercel** — despliegue automático:

- Cada PR genera **preview URL** para validación. - Cada merge a `main` despliega a **producción**. - Variables de entorno gestionadas en Vercel Dashboard (no en repo).

Rollback

- En caso de incidente: `vercel rollback` al deployment anterior conocido bueno (≤ 30 segundos).

6. Seguimiento del Reto (M6)

Responsable: Elvia

- **Sesiones de Reto** documentadas en actas internas del equipo.
- **Demos al socio formador** agendadas al cierre de cada sprint con la versión etiquetada de `main`.
- Retroalimentación del socio se traduce en issues nuevos, priorizados en planning del siguiente sprint.

Anexos

A. Plantilla de Pull Request

```
## Resumen
<qué cambia y por qué>

## Tickets / Issues
Closes #<id>

## Cambios principales
- ...

## Cómo probar
1. ...
2. ...

## Checklist
- [ ] CI en verde
- [ ] Pruebas añadidas/actualizadas
- [ ] Documentación actualizada (si aplica)
- [ ] Sin secretos commiteados
```

B. Plantilla de Issue de Bug

```
**Descripción:** ...  
**Pasos para reproducir:** 1. ... 2. ... 3. ...  
**Esperado:** ...  
**Observado:** ...  
**Versión / commit:** vX.Y.Z (sha ...)  
**Severidad:** crítica / alta / media / baja
```

C. Convención de mensajes de commit

Tipo: descripción breve en imperativo

Cuerpo opcional explicando qué/por qué.

Co-Authored-By: <colaborador> <correo>

Tipos aceptados: Add , Fix , Update , Refactor , UI , Docs , Chore , Test .

Fin del documento.